Deterministic Real-Time Audio Engineering on Android: Low-Level Optimization for the Samsung Exynos 2400 and Xclipse 940 Architecture

Delivering sub-2 ms deterministic audio processing on unrooted production Android devices requires coordinating Linux kernel thread scheduling, asymmetric multiprocessing (AMP) topologies, aggressive OEM background task mitigation, heterogeneous Vulkan compute pipelines, and SIMD vectorization. On the Samsung Galaxy S24+—powered by the Exynos 2400 (S5E9945) SoC and the AMD RDNA3-based Samsung Xclipse 940 GPU running Samsung One UI—achieving jitter-free, bit-perfect execution requires bypassing high-level framework layers while working strictly within unrooted security boundaries.

Exynos 2400 Core Topology, Thread Pinning, and Background Liveness under Samsung One UI

Microarchitectural Cluster Topology and Unrooted Scheduling Realities

The Exynos 2400 implements a 10-core, 4-cluster heterogeneous ARMv9.2-A compute complex fabricated on a 4 nm LPP+ node. Workload determinism requires mapping specific processing stages to the physical microarchitecture based on cache isolation and thermal dynamics.

Cluster Identity	Core Mask	Microarchitecture	Max Frequency	Private L2 Cache	Architectural Profile & Target Engine Mapping
Cluster 0 (LITTLE)	CPU 0–3	ARM Cortex-A520	1.95 GHz	512 KB	In-order efficiency cores; JNI bridges, Bluetooth telemetry, low-rate UI polling
Cluster 1 (Mid-Low)	CPU 4–6	ARM Cortex-A720	2.60 GHz	512 KB	Out-of-order performance cores; Asynchronous USB I/O pump (USBDEVFS_REAPURB)

Cluster Identity	Core Mask	Microarchitecture	Max Frequency	Private L2 Cache	Architectural Profile & Target Engine Mapping
Cluster 2 (Mid-High)	CPU 7–8	ARM Cortex-A720	2.90 GHz	512 KB	Sustained high-frequency cores; Master real-time audio DSP callback loop
Cluster 3 (Prime)	CPU 9	ARM Cortex-X4	3.21 GHz	2048 KB	Extreme single-thread burst core; Excluded from real-time audio threads

On stock production Android devices, the SELinux security domain and kernel cgroup rules strictly prevent unprivileged applications from elevating threads into real-time POSIX scheduling classes. Any call to sched_setscheduler() requesting SCHED_FIFO or SCHED_RR fails with EPERM. Consequently, audio engines must operate within the Completely Fair Scheduler (SCHED_NORMAL / CFS).

Deterministic execution under CFS is achieved through a multi-tiered configuration. Initiating thread promotion via android.os.Process.setThreadPriority(Process.THREAD_PRIORITY_URGENT_AUDIO) sets the thread's Linux nice level to -19. This causes the framework's resource manager to migrate the thread ID into the /dev/cpuset/audio-app or /dev/cpuset/top-app cgroup, maximizing CFS execution weight and minimizing preemption intervals during concurrent workloads.

To eliminate cross-cluster scheduling migration penalties and prevent cache thrashing, explicit CPU affinity masks must be bound using sched_setaffinity() or pthread_setaffinity_np(). The master DSP thread should be pinned to Cluster 2 (Cores 7 and 8), which consists of high-clocked Cortex-A720 cores. Cortex-A520 efficiency cores (Cores 0–3) must be excluded because their constrained dual-issue in-order execution pipeline cannot maintain sub-2 ms deadlines under complex filter loads.

The Cortex-X4 prime core (Core 9) must also be omitted. While offering peak single-threaded instructions-per-cycle (IPC), the high thermal dissipation of the Cortex-X4 induces steep Dynamic Voltage and Frequency Scaling (DVFS) thermal throttling, resulting in unpredictable clock reductions. Furthermore, its deep power-down transitions (C-states) incur entry and exit latencies exceeding 500\ \mu\text{s}, introducing unacceptable phase jitter into the audio callback.

Thread scheduling must be reinforced using the Android Dynamic Performance Framework (ADPF) via APerformanceHintManager and APerformanceHintSession in NDK android/performance_hint.h. By creating an ADPF session targeting the audio callback quantum and periodically reporting actual cycle durations using APerformanceHint_reportActualWorkDuration(), the audio engine directly informs the Energy Aware Scheduler (EAS) governor. This signals the governor to sustain Cluster 2 CPU operating frequencies, preventing sudden clock drops during low-complexity processing blocks.

Samsung One UI Background Power-Saving Defenses

Samsung One UI features aggressive background process management managed by com.samsung.android.lool (Samsung Device Care). When the device display turns off, One UI enforces power policies that can suspend user threads, throttle background cgroups, and terminate unexempted background processes within three minutes.
Maintaining uninterrupted processing requires satisfying three interconnected operational criteria. The host process must run an active Android ForegroundService associated with the system attribute foregroundServiceType="mediaPlayback", supported by a persistent status-bar notification. In parallel, an active PowerManager.PARTIAL_WAKE_LOCK must be acquired to prevent the SoC application processor from entering system suspend while the screen is unpowered.
Because Samsung Device Care places apps into "Sleeping" or "Deep Sleeping" states based on heuristic metrics, the application must prompt the user to register it within the "Never sleeping apps" whitelist. This registration is executed by launching a deep-link Intent targeting com.samsung.android.sm.ACTION_OPEN_CHECKABLE_LISTACTIVITY within package com.samsung.android.lool, configuring activity_type = 2. The app must also request exemption from standard Android Doze mode via Settings.ACTION_REQUEST_IGNORE_BATTERY_OPTIMIZATIONS.

Direct Native Asynchronous USBFS Processing Pipeline

Routing audio through Android's default audio HAL introduces system latency and potential buffer underruns during background operation. Bypassing this framework requires direct user-space communication with external USB Digital-to-Analog Converters (DACs) using Linux usbfs nodes accessed via /dev/bus/usb/XXX/YYY.
Isochronous audio streaming over usbfs requires double-buffered, asynchronous request-reap pipelines. The processing thread enqueues Isochronous USB Request Blocks (URBs) through ioctl(fd, USBDEVFS_SUBMITURB, &urb) and collects completed transfers via ioctl(fd, USBDEVFS_REAPURBNDELAY, &reaped_urb). Using the non-blocking reap variant ensures the thread never blocks inside the kernel driver.
When no completed URB is pending, the thread suspends execution by awaiting an event via epoll_wait() on the USB file descriptor with an exact timeout corresponding to the audio quantum. This allows the thread to yield execution cleanly to the scheduler while waking immediately upon hardware buffer completion, eliminating the latency and overhead of continuous polling.

```
#include <jni.h>
#include <pthread.h>
#include <sched.h>
#include <unistd.h>
#include <sys/ioctl.h>
#include <sys/epoll.h>
#include <linux/usbdevice_fs.h>
#include <android/performance_hint.h>
#include <chrono>
#include <vector>
#include <atomic>
```

```
#include <cstring>

class RealtimeAudioEngine {
public:
    RealtimeAudioEngine() : running_(false), usb_fd_(-1),
hint_session_(nullptr) {}

    bool InitializeADPF(int64_t target_work_duration_ns) {
        APerformanceHintManager* manager =
APerformanceHint_getManager();
        if (!manager) return false;

        pid_t tid = gettid();
        hint_session_ = APerformanceHint_createSession(manager, &tid,
1, target_work_duration_ns);
        return hint_session_ != nullptr;
    }

    void PinToCluster(int cluster_mask) {
        cpu_set_t cpuset;
        CPU_ZERO(&cpuset);
        for (int i = 0; i < 10; ++i) {
            if (cluster_mask & (1 << i)) {
                CPU_SET(i, &cpuset);
            }
        }
        pthread_setaffinity_np(pthread_self(), sizeof(cpu_set_t),
&cpuset);
    }

    void StartAudioPump(int usb_fd) {
        usb_fd_ = usb_fd;
        running_.store(true, std::memory_order_release);
        worker_thread_ =
std::jthread(&RealtimeAudioEngine::UsbPumpLoop, this);
    }

    void StopAudioPump() {
        running_.store(false, std::memory_order_release);
        if (worker_thread_.joinable()) {
            worker_thread_.join();
        }
        if (hint_session_) {
            APerformanceHint_closeSession(hint_session_);
            hint_session_ = nullptr;
        }
    }
```

```
private:
    void UsbPumpLoop() {
        // Pin to Exynos 2400 Cluster 2: Cores 7 and 8 (Cortex-A720
High: 2.90 GHz)
        // Bitmask: (1 << 7) | (1 << 8) = 0x80 | 0x100 = 0x180 (384)
        PinToCluster(0x180);

        constexpr int NUM_URBS = 4;
        constexpr int PACKETS_PER_URB = 8;
        constexpr int ISO_PACKET_SIZE = 192; // 24-bit 96kHz stereo
subframe quantum
        std::vector<std::vector<uint8_t>> buffers(
            NUM_URBS, std::vector<uint8_t>(PACKETS_PER_URB *
ISO_PACKET_SIZE));
        std::vector<usbdevfs_urb> urbs(NUM_URBS);
        std::vector<std::vector<usbdevfs_iso_packet_desc>> iso_descs(
            NUM_URBS,
std::vector<usbdevfs_iso_packet_desc>(PACKETS_PER_URB));

        for (int i = 0; i < NUM_URBS; ++i) {
            std::memset(&urbs[i], 0, sizeof(usbdevfs_urb));
            urbs[i].type = USBDEVFS_URB_TYPE_ISO;
            urbs[i].endpoint = 0x01; // Isochronous Out Endpoint 1
            urbs[i].buffer = buffers[i].data();
            urbs[i].buffer_length = buffers[i].size();
            urbs[i].number_of_packets = PACKETS_PER_URB;
            urbs[i].iso_frame_desc = iso_descs[i].data();

            for (int p = 0; p < PACKETS_PER_URB; ++p) {
                urbs[i].iso_frame_desc[p].length = ISO_PACKET_SIZE;
            }
            ioctl(usb_fd_, USBDEVFS_SUBMITURB, &urbs[i]);
        }

        int epoll_fd = epoll_create1(EPOLL_CLOEXEC);
        epoll_event ev{};
        ev.events = EPOLLOUT;
        ev.data.fd = usb_fd_;
        epoll_ctl(epoll_fd, EPOLL_CTL_ADD, usb_fd_, &ev);

        while (running_.load(std::memory_order_relaxed)) {
            auto start_time = std::chrono::steady_clock::now();

            usbdevfs_urb* reaped_urb = nullptr;
            while (ioctl(usb_fd_, USBDEVFS_REAPURBNDELAY, &reaped_urb)
== 0) {
                if (reaped_urb) {
                    SynthesizeNextQuantum(
```

```
                        static_cast<uint8_t*>(reaped_urb->buffer),
                        reaped_urb->buffer_length);
                    ioctl(usb_fd_, USBDEVFS_SUBMITURB, reaped_urb);
                }
            }

            epoll_event events[1];
            epoll_wait(epoll_fd, events, 1, 1); // 1 ms bounded
fallback

            auto end_time = std::chrono::steady_clock::now();
            int64_t actual_duration_ns =
std::chrono::duration_cast<std::chrono::nanoseconds>(
                end_time - start_time).count();

            if (hint_session_) {

APerformanceHint_reportActualWorkDuration(hint_session_,
actual_duration_ns);
            }
        }

        close(epoll_fd);
    }

    void SynthesizeNextQuantum(uint8_t* buffer, size_t len) {
        std::memset(buffer, 0, len);
    }

    std::atomic<bool> running_;
    int usb_fd_;
    APerformanceHintSession* hint_session_;
    std::jthread worker_thread_;
};

package com.engine.audio

import android.app.Notification
import android.app.NotificationChannel
import android.app.NotificationManager
import android.app.Service
import android.content.Context
import android.content.Intent
import android.content.pm.ServiceInfo
import android.net.Uri
import android.os.Build
import android.os.IBinder
import android.os.PowerManager
```

```
import android.os.Process
import android.provider.Settings

class DeterministicAudioService : Service() {
    private var wakeLock: PowerManager.WakeLock? = null

    override fun onCreate() {
        super.onCreate()

Process.setThreadPriority(Process.THREAD_PRIORITY_URGENT_AUDIO)

        val powerManager = getSystemService(Context.POWER_SERVICE) as
PowerManager
        wakeLock = powerManager.newWakeLock(
            PowerManager.PARTIAL_WAKE_LOCK,
            "RealtimeAudioEngine::PlaybackLock"
        ).apply {
            acquire(24 * 60 * 60 * 1000L)
        }

        startForegroundWithNotification()
        ensureSamsungUnrestricted()
    }

    private fun startForegroundWithNotification() {
        val channelId = "audio_playback_channel"
        val manager = getSystemService(Context.NOTIFICATION_SERVICE)
as NotificationManager
        val channel = NotificationChannel(
            channelId,
            "Realtime Audio Stream",
            NotificationManager.IMPORTANCE_LOW
        )
        manager.createNotificationChannel(channel)

        val notification = Notification.Builder(this, channelId)
            .setContentTitle("Deterministic Audio Core Active")
            .setContentText("Direct USBFS Isochronous streaming
active")
            .setOngoing(true)
            .build()

        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.Q) {
            startForeground(
                1001,
                notification,
                ServiceInfo.FOREGROUND_SERVICE_TYPE_MEDIA_PLAYBACK
            )
```

```
        } else {
            startForeground(1001, notification)
        }
    }

    private fun ensureSamsungUnrestricted() {
        val powerManager = getSystemService(Context.POWER_SERVICE) as
PowerManager
        if (!powerManager.isIgnoringBatteryOptimizations(packageName))
{
            val intent =
Intent(Settings.ACTION_REQUEST_IGNORE_BATTERY_OPTIMIZATIONS).apply {
                data = Uri.parse("package:$packageName")
                flags = Intent.FLAG_ACTIVITY_NEW_TASK
            }
            startActivity(intent)
        }

        try {
            val samsungIntent = Intent().apply {
                action =
"com.samsung.android.sm.ACTION_OPEN_CHECKABLE_LISTACTIVITY"
                setPackage("com.samsung.android.lool")
                putExtra("activity_type", 2) // 2: Never sleeping apps
                flags = Intent.FLAG_ACTIVITY_NEW_TASK
            }
            startActivity(samsungIntent)
        } catch (_: Exception) {
            // Non-Samsung fallback or modified One UI vendor binary
        }
    }

    override fun onBind(intent: Intent?): IBinder? = null

    override fun onDestroy() {
        wakeLock?.let { if (it.isHeld) it.release() }
        super.onDestroy()
    }
}
```

Sub-2ms Vulkan Compute Audio DSP on the AMD RDNA3 Xclipse 940 GPU

Architectural Profile of the Samsung Xclipse 940

The Samsung Xclipse 940 GPU is built upon AMD's RDNA3 microarchitecture, featuring 6 Workgroup Processors (WGPs) housing 12 Dual-Compute Units, which provide 768 stream processors clocked up to 1095 MHz. The processor interfaces directly with system memory over a 128-bit LPDDR5X bus delivering up to 68.2\text{ GB/s} of unified memory bandwidth.

Architectural Feature	Hardware Specification	Real-Time DSP Implication
Compute Units (CUs)	12 Dual-Compute Units (768 ALUs)	Parallel execution capability across multiple filter partitions
Wavefront Execution Width	Native Wave32 (32 threads per wavefront)	Workgroup sizes must be exact multiples of 32 to eliminate divergence
ALU Microarchitecture	Dual-Issue Mathematical Pipelines	High-throughput complex multiply-accumulate operations
System Memory Hierarchy	Unified Physical Memory Architecture (UMA)	Eliminates discrete PCIe-style transfers via zero-copy host buffers
Peak GPU Compute Throughput	3406.8 GFLOPS (FP32)	Accommodates massive multi-partition convolution under 2 ms

The native Wave32 execution mode groups shader invocations into tightly coupled wavefronts of 32 threads. Configuring compute shader workgroups as multiples of 32 (such as local_size_x = 64 or 128) matches the hardware scheduling granularity, preventing execution-mask serialization and maximizing ALU occupancy across the compute units.
Because the Exynos 2400 integrates the CPU and GPU onto a single silicon die sharing LPDDR5X RAM, physical memory copies are unnecessary if virtual allocations are configured correctly.

Zero-Copy Interop: Direct Host Coherence vs. AHardwareBuffer

Zero-copy GPU acceleration can be structured through two primary paths on Android. The first approach relies on native Vulkan buffers allocated with the memory properties VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT | VK_MEMORY_PROPERTY_HOST_COHERENT_BIT. By persistently mapping this memory region via vkMapMemory(), the CPU callback writes audio samples directly into memory accessible by the GPU. On the unified memory architecture of the Exynos 2400, cache-coherency transactions are handled directly by the system interconnect fabric, eliminating the overhead of manual memory cache invalidation (vkFlushMappedMemoryRanges()). This path provides minimal latency within a single operating process.
The alternative approach uses AHardwareBuffer from the Android NDK coupled with the extension VK_ANDROID_external_memory_android_hardware_buffer. Allocating an AHardwareBuffer configured with AHARDWAREBUFFER_USAGE_GPU_DATA_BUFFER | AHARDWAREBUFFER_USAGE_CPU_READ_OFTEN | AHARDWAREBUFFER_USAGE_CPU_WRITE_OFTEN allows the underlying physical pages to be imported directly into Vulkan as a VkBuffer or VkImage. While introducing slightly more driver indirection than a native coherent VkBuffer, AHardwareBuffer enables zero-copy buffer sharing across process boundaries via Binder or UNIX domain sockets, which is useful when isolating the DSP compute pipeline into an external helper service.

Uniformly Partitioned Overlap-Save FFT Convolution (131,072 Taps)

Direct frequency-domain convolution of an impulse response spanning N = 131,072 samples introduces prohibitive latency if executed as a single monolithic Fast Fourier Transform (FFT). At a 96 kHz sampling rate, an unpartitioned block would produce over 1.36\text{ seconds} of algorithmic delay.
Uniformly Partitioned Overlap-Save (UPOLS) convolution resolves this latency issue by dividing the 131,072-tap filter into P smaller uniform sub-blocks of length B. The algorithmic input-to-output latency is governed entirely by the partition size B. To operate within a strict <2\text{ ms} processing deadline at 96 kHz, the block size is set to B = 128 samples (1.33\text{ ms} algorithmic latency), generating P = 1024 frequency-domain partitions:
P = \frac{N}{B} = \frac{131072}{128} = 1024\text{ partitions}
Each real-time callback iteration processes a time-domain block of 2B = 256 samples, comprising the previous B input samples overlapping the current B incoming samples.
The pipeline proceeds through four sequential mathematical stages:

1. An in-place Forward Complex Radix-2 FFT transforms the 2B time-domain window into the frequency-domain spectrum X_k, using the GPU's on-chip Local Data Store (LDS) for intermediate staging.
2. The resulting spectrum X_k is written to a circular history buffer maintaining P historical spectral frames.
3. A Frequency-Domain Multiply-Accumulate (FD-MAC) accumulation step computes the inner dot-product across all P partitions against the pre-transformed impulse response partitions H_p:
   Y_k = \sum_{p=0}^{P-1} X_{(k - p) \bmod P} \odot H_p
1. An Inverse Complex FFT (IFFT) converts the accumulated spectrum Y_k back to the time domain.

The first B samples of the output represent circular convolution time-aliasing and are discarded; the remaining B samples constitute the filtered audio signal.

Bounded Command Dispatch and Synchronization

Maintaining a deterministic <2\text{ ms} execution budget requires avoiding CPU-blocking functions such as vkQueueWaitIdle() or vkWaitForFences(). In their place, the engine pre-records all Vulkan compute commands into a static command buffer during initialization. During the real-time audio callback, the CPU writes input audio into the persistently mapped coherent host buffer, updates a dynamic circular index via Vulkan push constants, and issues a non-blocking vkQueueSubmit(). Synchronization is coordinated asynchronously using Timeline Semaphores (VK_KHR_timeline_semaphore). The host checks execution progress non-blockingly via vkGetSemaphoreCounterValue(), preventing driver synchronization stalls on the audio thread.

```
#version 450
layout(local_size_x = 64, local_size_y = 1, local_size_z = 1) in;

#define FFT_SIZE 256
#define HALF_SIZE 128
#define NUM_PARTITIONS 1024
```

```
struct Complex {
    float r;
    float i;
};

Complex complex_mul(Complex a, Complex b) {
    return Complex(a.r * b.r - a.i * b.i, a.r * b.i + a.i * b.r);
}

layout(std430, binding = 0) readonly buffer InputTimeDomain {
    float in_samples[];
};

layout(std430, binding = 1) readonly buffer PartitionIRFDL {
    Complex ir_partitions[]; // Packed: [NUM_PARTITIONS][FFT_SIZE]
};

layout(std430, binding = 2) buffer FDLHistory {
    Complex fdl_history[]; // Circular Ring:
[NUM_PARTITIONS][FFT_SIZE]
};

layout(std430, binding = 3) writeonly buffer OutputTimeDomain {
    float out_samples[];
};

layout(push_constant) uniform PushConstants {
    uint write_idx; // Current index in circular history ring
} push;

shared Complex lds_scratch[FFT_SIZE];

void main() {
    uint tid = gl_LocalInvocationID.x;
    uint total_threads = gl_WorkGroupSize.x;

    // Load time-domain samples into shared memory
    for (uint i = tid; i < FFT_SIZE; i += total_threads) {
        lds_scratch[i].r = in_samples[i];
        lds_scratch[i].i = 0.0;
    }
    barrier();

    // In-place Radix-2 Forward FFT
    for (uint stride = FFT_SIZE / 2; stride > 0; stride /= 2) {
        for (uint i = tid; i < FFT_SIZE / 2; i += total_threads) {
            uint j = ((i / stride) * (stride * 2)) + (i % stride);
            Complex u = lds_scratch[j];
```

```
            Complex v = lds_scratch[j + stride];
            float angle = -2.0 * 3.14159265358979323846 * float(i %
stride) / float(stride * 2);
            Complex twiddle = Complex(cos(angle), sin(angle));
            Complex v_twiddled = complex_mul(v, twiddle);

            lds_scratch[j] = Complex(u.r + v_twiddled.r, u.i +
v_twiddled.i);
            lds_scratch[j + stride] = Complex(u.r - v_twiddled.r, u.i
- v_twiddled.i);
        }
        barrier();
    }

    // Write computed spectrum to circular history buffer
    for (uint i = tid; i < FFT_SIZE; i += total_threads) {
        fdl_history[push.write_idx * FFT_SIZE + i] = lds_scratch[i];
    }
    barrier();

    // Uniformly Partitioned Frequency-Domain Multiply-Accumulate
    for (uint bin = tid; bin < FFT_SIZE; bin += total_threads) {
        Complex accum = Complex(0.0, 0.0);
        for (uint p = 0; p < NUM_PARTITIONS; ++p) {
            uint hist_idx = (push.write_idx + NUM_PARTITIONS - p) %
NUM_PARTITIONS;
            Complex X = fdl_history[hist_idx * FFT_SIZE + bin];
            Complex H = ir_partitions[p * FFT_SIZE + bin];
            Complex prod = complex_mul(X, H);
            accum.r += prod.r;
            accum.i += prod.i;
        }
        lds_scratch[bin] = accum;
    }
    barrier();

    // In-place Inverse FFT (IFFT)
    for (uint stride = 1; stride < FFT_SIZE; stride *= 2) {
        for (uint i = tid; i < FFT_SIZE / 2; i += total_threads) {
            uint j = ((i / stride) * (stride * 2)) + (i % stride);
            Complex u = lds_scratch[j];
            Complex v = lds_scratch[j + stride];
            float angle = 2.0 * 3.14159265358979323846 * float(i %
stride) / float(stride * 2);
            Complex twiddle = Complex(cos(angle), sin(angle));
            Complex v_twiddled = complex_mul(v, twiddle);

            lds_scratch[j] = Complex(u.r + v_twiddled.r, u.i +
```

```
v_twiddled.i);
            lds_scratch[j + stride] = Complex(u.r - v_twiddled.r, u.i
- v_twiddled.i);
        }
        barrier();
    }

    // Discard aliased overlap (first HALF_SIZE samples), export
normalized signal
    for (uint i = tid; i < HALF_SIZE; i += total_threads) {
        out_samples[i] = lds_scratch[i + HALF_SIZE].r /
float(FFT_SIZE);
    }
}

#version 450
layout(local_size_x = 64, local_size_y = 1, local_size_z = 1) in;

#define TAPS_PER_PHASE 32
#define DECIMATION_FACTOR 2

layout(std430, binding = 0) readonly buffer HighRateInput {
    float in_samples[];
};

layout(std430, binding = 1) readonly buffer PolyphaseCoefficients {
    // Coefficients arranged as [DECIMATION_FACTOR][TAPS_PER_PHASE]
    float subfilter_taps[];
};

layout(std430, binding = 2) writeonly buffer DecimatedOutput {
    float out_samples[];
};

layout(push_constant) uniform PolyphaseParams {
    uint num_output_samples;
} params;

void main() {
    uint out_idx = gl_GlobalInvocationID.x;
    if (out_idx >= params.num_output_samples) return;

    float acc = 0.0;
    uint input_anchor = out_idx * DECIMATION_FACTOR;

    for (uint tap = 0; tap < TAPS_PER_PHASE; ++tap) {
        acc += in_samples[input_anchor - tap] * subfilter_taps[tap];
    }
```

```
    out_samples[out_idx] = acc;
}
```

Lock-Free "Memory Play" RAM Buffer and 24-Bit Forensic Bit-Masking

Linux Memory Locking Constraints and Page Fault Elimination

Audiophile-grade "Memory Play" engines pre-load decoded uncompressed audio streams into system RAM to prevent flash storage bus contention during real-time playback. However, pinning large memory buffers using mlock() fails on standard Android devices due to user-space security limits:

```
struct rlimit rl;
getrlimit(RLIMIT_MEMLOCK, &rl);
// On standard unrooted Android: rl.rlim_cur = 65536 bytes (64 KB)
```

Because RLIMIT_ME[span_21](start_span)[span_21](end_span)MLOCK is capped at 64\text{ KB} for unprivileged processes, invoking mlock() on a 256\text{ MB} allocation fails immediately with ENOMEM or EPERM.
To achieve zero page faults across playback without requiring root privileges, the allocation strategy must leverage Linux kernel demand-paging semantics:

- **Anonymous Memory Mapping**: The memory block is allocated using mmap() configured with MAP_PRIVATE | MAP_ANONYMOUS | MAP_POPULATE.
- **Kernel Paging Directives**: Applying madvise(addr, size, MADV_WILLNEED) instructs the kernel to pre-fault the entire virtual memory range into active physical memory tables. Complementing this with madvise(addr, size, MADV_DONTFORK) prevents unnecessary copy-on-write page table duplications if helper threads or sub-processes are spawned.
- **Proactive Page Touching**: Walking the allocated address space at system page boundaries (4096 bytes) and writing a zero value forces the Linux page management subsystem to allocate physical page frames and populate the ARM Translation Lookaside Buffer (TLB).

Coupling this pre-faulting procedure with an active Android ForegroundService and Process.THREAD_PRIORITY_URGENT_AUDIO assigns the host process an Out-Of-Memory (OOM) score adjustment of 0. This protects the pinned pages from eviction by the Low Memory Killer Daemon (LMKD) or migration into swap/zRAM.

Cache-Line Isolated C++20 SPSC Lock-Free Audio Ring Buffer

To pass audio samples between asynchronous file decoding, GPU compute passes, and the real-time USB callback thread without mutex locks or priority inversions, the engine uses a Single-Producer Single-Consumer (SPSC) circular queue. Producer and consumer indices are aligned to 64\text{ bytes} (the ARM Cortex-A720 cache line width) to eliminate false sharing between CPU cores.

```
#include <atomic>
```

```
#include <cstddef>
#include <new>
#include <vector>
#include <span>
#include <algorithm>

template <typename T, size_t Capacity>
class LockFreeAudioRingBuffer {
    static_assert((Capacity & (Capacity - 1)) == 0, "Capacity must be
a power of two");
    static constexpr size_t CACHE_LINE_SIZE = 64;

public:
    LockFreeAudioRingBuffer() : head_(0), tail_(0) {
        ring_buffer_ = static_cast<T*>(::operator new[](
            sizeof(T) * Capacity, std::align_val_t{alignof(T)}));
    }

    ~LockFreeAudioRingBuffer() {
        ::operator delete[](ring_buffer_,
std::align_val_t{alignof(T)});
    }

    size_t Write(std::span<const T> input) {
        const size_t tail = tail_.load(std::memory_order_relaxed);
        const size_t head = head_.load(std::memory_order_acquire);
        const size_t available = Capacity - (tail - head);
        const size_t to_write = std::min(input.size(), available);

        const size_t mask = Capacity - 1;
        const size_t part1 = std::min(to_write, Capacity - (tail &
mask));
        std::copy_n(input.data(), part1, ring_buffer_ + (tail &
mask));
        std::copy_n(input.data() + part1, to_write - part1,
ring_buffer_);

        tail_.store(tail + to_write, std::memory_order_release);
        return to_write;
    }

    size_t Read(std::span<T> output) {
        const size_t head = head_.load(std::memory_order_relaxed);
        const size_t tail = tail_.load(std::memory_order_acquire);
        const size_t available = tail - head;
        const size_t to_read = std::min(output.size(), available);

        const size_t mask = Capacity - 1;
```

```
        const size_t part1 = std::min(to_read, Capacity - (head &
mask));
        std::copy_n(ring_buffer_ + (head & mask), part1,
output.data());
        std::copy_n(ring_buffer_, to_read - part1, output.data() +
part1);

        head_.store(head + to_read, std::memory_order_release);
        return to_read;
    }

private:
    alignas(CACHE_LINE_SIZE) std::atomic<size_t> tail_;
    alignas(CACHE_LINE_SIZE) std::atomic<size_t> head_;
    alignas(CACHE_LINE_SIZE) T* ring_buffer_;
};
```

ARM64 NEON Forensic Signal Analysis: LSB Activity and Ultrasonic Auditing

Commercial high-resolution distributions occasionally contain artificial upsamplings. Common defects include padding 16-bit master files into 24-bit integer containers or upsampling standard 44.1 kHz material to 96 kHz/192 kHz without genuine acoustic energy above the Nyquist cutoff of 22.05 kHz.
The following ARM64 NEON routine validates bit-depth authenticity and flags artificial upsampling in real time within the callback loop:

```
#include <arm_neon.h>
#include <cstdint>
#include <cstddef>
#include <span>

struct AudioForensicMetrics {
    uint32_t active_bits_mask;
    bool has_true_24bit_dynamics;
    bool is_fake_upsampled;
    float ultrasonic_to_baseband_ratio;
};

class ForensicStreamAuditor {
public:
    static AudioForensicMetrics AuditFrame(
        std::span<const int32_t> pcm_samples, float sample_rate_hz) {

        AudioForensicMetrics metrics{};
        const size_t count = pcm_samples.size();
        const int32_t* ptr = pcm_samples.data();
```

```
        uint32x4_t mask_accum = vdupq_n_u32(0);
        float32x4_t v_energy_total = vdupq_n_f32(0.0f);
        float32x4_t v_energy_ultra = vdupq_n_f32(0.0f);

        // High-pass FIR taps configured for ~22.05 kHz corner
frequency (at 96 kHz)
        const float32x4_t tap_center = vdupq_n_f32(0.8f);
        const float32x4_t tap_side1  = vdupq_n_f32(-0.1f);
        const float32x4_t tap_side2  = vdupq_n_f32(-0.05f);

        size_t i = 0;
        for (; i + 4 <= count; i += 4) {
            int32x4_t samples = vld1q_s32(ptr + i);
            uint32x4_t raw_bits = vreinterpretq_u32_s32(samples);

            mask_accum = vorrq_u32(mask_accum, raw_bits);

            float32x4_t f_samples = vcvtq_f32_s32(samples);
            v_energy_total = vfmaq_f32(v_energy_total, f_samples,
f_samples);

            if (i >= 4 && i + 8 <= count) {
                float32x4_t sm2 = vcvtq_f32_s32(vld1q_s32(ptr + i -
2));
                float32x4_t sm1 = vcvtq_f32_s32(vld1q_s32(ptr + i -
1));
                float32x4_t sp1 = vcvtq_f32_s32(vld1q_s32(ptr + i +
1));
                float32x4_t sp2 = vcvtq_f32_s32(vld1q_s32(ptr + i +
2));

                float32x4_t high_passed = vmulq_f32(f_samples,
tap_center);
                high_passed = vfmaq_f32(high_passed, vaddq_f32(sm1,
sp1), tap_side1);
                high_passed = vfmaq_f32(high_passed, vaddq_f32(sm2,
sp2), tap_side2);

                v_energy_ultra = vfmaq_f32(v_energy_ultra,
high_passed, high_passed);
            }
        }

        uint32_t folded_mask = vgetq_lane_u32(mask_accum, 0) |
                               vgetq_lane_u32(mask_accum, 1) |
                               vgetq_lane_u32(mask_accum, 2) |
                               vgetq_lane_u32(mask_accum, 3);
```

```
        metrics.active_bits_mask = folded_mask;

        // For 24-bit audio aligned within a 32-bit container, true
24-bit dynamics
        // exhibit activity across the lower 8 bits (mask:
0x000000FF).
        metrics.has_true_24bit_dynamics = (folded_mask & 0xFF) != 0;

        float total_energy = vaddvq_f32(v_energy_total);
        float ultra_energy = vaddvq_f32(v_energy_ultra);

        metrics.ultrasonic_to_baseband_ratio = (total_energy > 0.0f) ?
            (ultra_energy / total_energy) : 0.0f;

        // If the sample rate is >= 88.2 kHz, genuine high-resolution
material exhibits
        // harmonic energy or dither noise above 22.05 kHz. An
ultrasonic ratio dropping
        // below -90 dB indicates brickwall truncation from upsampled
standard-rate content.
        metrics.is_fake_upsampled = (sample_rate_hz >= 88200.0f) &&

(metrics.ultrasonic_to_baseband_ratio < 1e-9f);

        return metrics;
    }
};
```

Android Unrooted Bluetooth A2DP Codec Telemetry and Dynamic Bitrate Extraction

Bluetooth A2DP Profile Architecture Across API Levels 34–36

Accessing low-level Bluetooth hardware parameters on unrooted Android devices is regulated by Android framework boundaries. While applications cannot inspect kernel-level baseband drivers directly, applications with android.permission.BLUETOOTH_CONNECT can query real-time transport states via BluetoothA2dp proxies through BluetoothProfile.ServiceListener. The structure of the query pipeline depends on the platform version:

- On Android 14 (API 34), runtime states are obtained via BluetoothA2dp.getCodecStatus(device).
- On Android 15 (API 35) and Android 16 (API 36), the platform introduced BluetoothCodecType via BluetoothA2dp.getSupportedCodecTypes(). Active session telemetry continues to be evaluated via BluetoothCodecStatus.getCodecConfig().

Telemetry Field	Target Class / API Method	Bitmask Representation / Parsing Details
Active Codec Type	BluetoothCodecConfig.getCodecType()	SOURCE_CODEC_TYPE_LDAC (4) SO[span_49](start_span)[span_49](end_span)[span_52](start_span)[span_52](end_span)URCE_CODEC_TYPE_INVALID (1,000,000 / Vendor fallback)
Active Sample Rate	BluetoothCodecConfig.getSampleRate()	SAMPLE_RATE_44100 (0x1), SAMPLE_RATE_48000 (0x2), SAMPLE_RATE_96000 (0x8), SAMPLE_RATE_192000 (0x10)
Active Bit Depth	BluetoothCodecConfig.getBitsPerSample()	BITS_PER_SAMPLE_16 (0x1), BITS_PER_SAMPLE_24 (0x2), BITS_PER_SAMPLE_32 (0x4)
LDAC Target Bitrate	BluetoothCodecConfig.getCodecSpecific1()	Evaluates bits 0–15: 1000 (990\text{ kbps}), 1001 (660\text{ kbps}), 1002 (330\text{ kbps})
Samsung Seamless	BluetoothCodecType.getCodecName() / getCodecId()	Vendor ID 0x0075; Codec Name string matching "Samsung Seamless Codec" or "SSC-UHQ"

Deconstructing Real-Time Sony LDAC Bitrate Stepping

The Sony LDAC encoder incorporates dynamic Adaptive Bitrate (ABR) capabilities to preserve audio continuity when 2.4 GHz RF link budgets degrade. The active transmission rate is not exposed through standard bitrate accessor methods, but is embedded within the lower 16 bits of BluetoothCodecConfig.getCodecSpecific1():

CodecSpecific1 Value	Operating Mode	Transmission Bitrate	Audio Fidelity Profile
1000 (0x3E8)	High Quality (HQ)	990\text{ kbps}	Full 24-bit/96kHz high-resolution transmission
1001 (0x3E9)	Standard Quality (SQ)	660\text{ kbps}	Intermediate acoustic resolution
1002 (0x3EA)	Mobile Quality (MQ)	330\text{ kbps}	Prioritizes connection stability over spectral fidelity
1003 (0x3EB)	Dynamic ABR	Variable (330–990 kbps)	Real-time rate scaling driven by link packet drops

Monitoring getCodecSpecific1() allows the native engine to detect when local RF interference forces LDAC down-stepping to 330 kbps. The engine can then adjust its internal DSP complexity or advise the user of reduced transmission fidelity.

Resolving Samsung Seamless Codec (SSC / SSC-UHQ) Status

The Exynos 2400 platform supports Samsung Seamless Codec (SSC), including its ultra-high-definition variant SSC-UHQ (up to 24-bit / 96 kHz or 192 kHz). In standard AOSP code bases, SSC operates as an external vendor-specific profile:

- When queried via standard legacy enumerations, BluetoothCodecConfig.getCodecType() returns SOURCE_CODEC_TYPE_INVALID (1,000,000 / 0xF4240).
- Under API levels 35 and 36, querying the extended structure via BluetoothCodecType.getCodecName() yields "Samsung Sea[span_62](start_span)[span_62](end_span)[span_64](start_span)[span_64](end_span)[span_66](start_span)[span_66](end_span)mless Codec" or "SSC-UHQ".
- The 40-bit composite identifier from BluetoothCodecType.getCodecId() contains Samsung's Bluetooth SIG Vendor Identifier (0x0075) in its upper bits.

```
package com.engine.audio.bluetooth

import android.annotation.SuppressLint
import android.bluetooth.BluetoothA2dp
import android.bluetooth.BluetoothAdapter
import android.bluetooth.BluetoothCodecConfig
import android.bluetooth.BluetoothCodecStatus
import android.bluetooth.BluetoothDevice
import android.bluetooth.BluetoothManager
import android.bluetooth.BluetoothProfile
import android.content.BroadcastReceiver
import android.content.Context
import android.content.Intent
import android.content.IntentFilter
import android.os.Build

class BluetoothTelemetryManager(
    private val context: Context,
    private val nativeEnginePtr: Long
) {
    private val bluetoothManager =
context.getSystemService(Context.BLUETOOTH_SERVICE) as
BluetoothManager
    private val bluetoothAdapter: BluetoothAdapter? =
bluetoothManager.adapter
    private var a2dpProfile: BluetoothA2dp? = null

    private val profileListener = object :
BluetoothProfile.ServiceListener {
        override fun onServiceConnected(profile: Int, proxy:
BluetoothProfile) {
            if (profile == BluetoothProfile.A2DP) {
                a2dpProfile = proxy as BluetoothA2dp
                pollActiveCodecStatus()
```

```
            }
        }

        override fun onServiceDisconnected(profile: Int) {
            if (profile == BluetoothProfile.A2DP) {
                a2dpProfile = null
            }
        }
    }

    private val codecChangeReceiver = object : BroadcastReceiver() {
        override fun onReceive(ctx: Context, intent: Intent) {
            if (intent.action ==
BluetoothA2dp.ACTION_CODEC_CONFIG_CHANGED) {
                pollActiveCodecStatus()
            }
        }
    }

    @SuppressLint("MissingPermission")
    fun initialize() {
        bluetoothAdapter?.getProfileProxy(context, profileListener,
BluetoothProfile.A2DP)
        val filter =
IntentFilter(BluetoothA2dp.ACTION_CODEC_CONFIG_CHANGED)
        context.registerReceiver(codecChangeReceiver, filter)
    }

    @SuppressLint("MissingPermission")
    fun pollActiveCodecStatus() {
        val a2dp = a2dpProfile ?: return
        val activeDevice: BluetoothDevice = a2dp.activeDevice ?:
return
        val codecStatus: BluetoothCodecStatus =
a2dp.getCodecStatus(activeDevice) ?: return
        val config: BluetoothCodecConfig = codecStatus.codecConfig ?:
return

        val sampleRate = parseSampleRate(config.sampleRate)
        val bitDepth = parseBitDepth(config.bitsPerSample)
        val codecType = config.codecType
        val specific1 = config.codecSpecific1

        var actualLdacBitrate = 0
        var isSamsungSeamless = false

        if (codecType == BluetoothCodecConfig.SOURCE_CODEC_TYPE_LDAC)
{
```

```
            actualLdacBitrate = when (specific1.toInt() and 0xFFFF) {
                1000 -> 990
                1001 -> 660
                1002 -> 330
                else -> 0 // Dynamic ABR mode active
            }
        }

        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
            val extendedType = config.extendedCodecType
            if (extendedType?.codecName?.contains("Samsung",
ignoreCase = true) == true ||
                extendedType?.codecId == 0x0075L) {
                isSamsungSeamless = true
            }
        } else {
            if (codecType == 1000000 || codecType == 11) {
                isSamsungSeamless = true
            }
        }

        nativePushCodecTelemetry(
            nativeEnginePtr,
            codecType,
            sampleRate,
            bitDepth,
            actualLdacBitrate,
            isSamsungSeamless
        )
    }

    private fun parseSampleRate(mask: Int): Int = when (mask) {
        BluetoothCodecConfig.SAMPLE_RATE_44100 -> 44100
        BluetoothCodecConfig.SAMPLE_RATE_48000 -> 48000
        BluetoothCodecConfig.SAMPLE_RATE_88200 -> 88200
        BluetoothCodecConfig.SAMPLE_RATE_96000 -> 96000
        BluetoothCodecConfig.SAMPLE_RATE_192000 -> 192000
        else -> 0
    }

    private fun parseBitDepth(mask: Int): Int = when (mask) {
        BluetoothCodecConfig.BITS_PER_SAMPLE_16 -> 16
        BluetoothCodecConfig.BITS_PER_SAMPLE_24 -> 24
        BluetoothCodecConfig.BITS_PER_SAMPLE_32 -> 32
        else -> 0
    }

    fun release() {
```

```
        try {
            context.unregisterReceiver(codecChangeReceiver)
        } catch (_: Exception) {}
        bluetoothAdapter?.closeProfileProxy(BluetoothProfile.A2DP,
a2dpProfile)
    }

    private external fun nativePushCodecTelemetry(
        enginePtr: Long,
        codecType: Int,
        sampleRate: Int,
        bitDepth: Int,
        ldacBitrate: Int,
        isSsc: Boolean
    )
}

#include <jni.h>
#include <android/log.h>
#include <atomic>
#include <cstdint>

#define TAG "AudioTelemetryJNI"

struct DynamicA2dpTelemetry {
    std::atomic<int32_t> codec_type{0};
    std::atomic<int32_t> sample_rate{0};
    std::atomic<int32_t> bit_depth{0};
    std::atomic<int32_t> ldac_bitrate_kbps{0};
    std::atomic<bool> is_samsung_seamless{false};
};

static DynamicA2dpTelemetry g_telemetry;

extern "C" JNIEXPORT void JNICALL
Java_com_engine_audio_bluetooth_BluetoothTelemetryManager_nativePushCo
decTelemetry(
    JNIEnv* /* env */,
    jobject /* thiz */,
    jlong /* enginePtr */,
    jint codecType,
    jint sampleRate,
    jint bitDepth,
    jint ldacBitrate,
    jboolean isSsc) {

    g_telemetry.codec_type.store(codecType,
std::memory_order_relaxed);
```

```
    g_telemetry.sample_rate.store(sampleRate,
std::memory_order_relaxed);
    g_telemetry.bit_depth.store(bitDepth, std::memory_order_relaxed);
    g_telemetry.ldac_bitrate_kbps.store(ldacBitrate,
std::memory_order_relaxed);
    g_telemetry.is_samsung_seamless.store(isSsc,
std::memory_order_release);

    __android_log_print(ANDROID_LOG_INFO, TAG,
        "Telemetry Update: Codec=%d, Rate=%d Hz, Depth=%d bit, LDAC
Bitrate=%d kbps, SSC=%s",
        codecType, sampleRate, bitDepth, ldacBitrate, isSsc ? "TRUE" :
"FALSE");
}

DynamicA2dpTelemetry SnapshotTelemetry() {
    return {
        g_telemetry.codec_type.load(std::memory_order_relaxed),
        g_telemetry.sample_rate.load(std::memory_order_relaxed),
        g_telemetry.bit_depth.load(std::memory_order_relaxed),
        g_telemetry.ldac_bitrate_kbps.load(std::memory_order_relaxed),

g_telemetry.is_samsung_seamless.load(std::memory_order_acquire)
    };
}
```

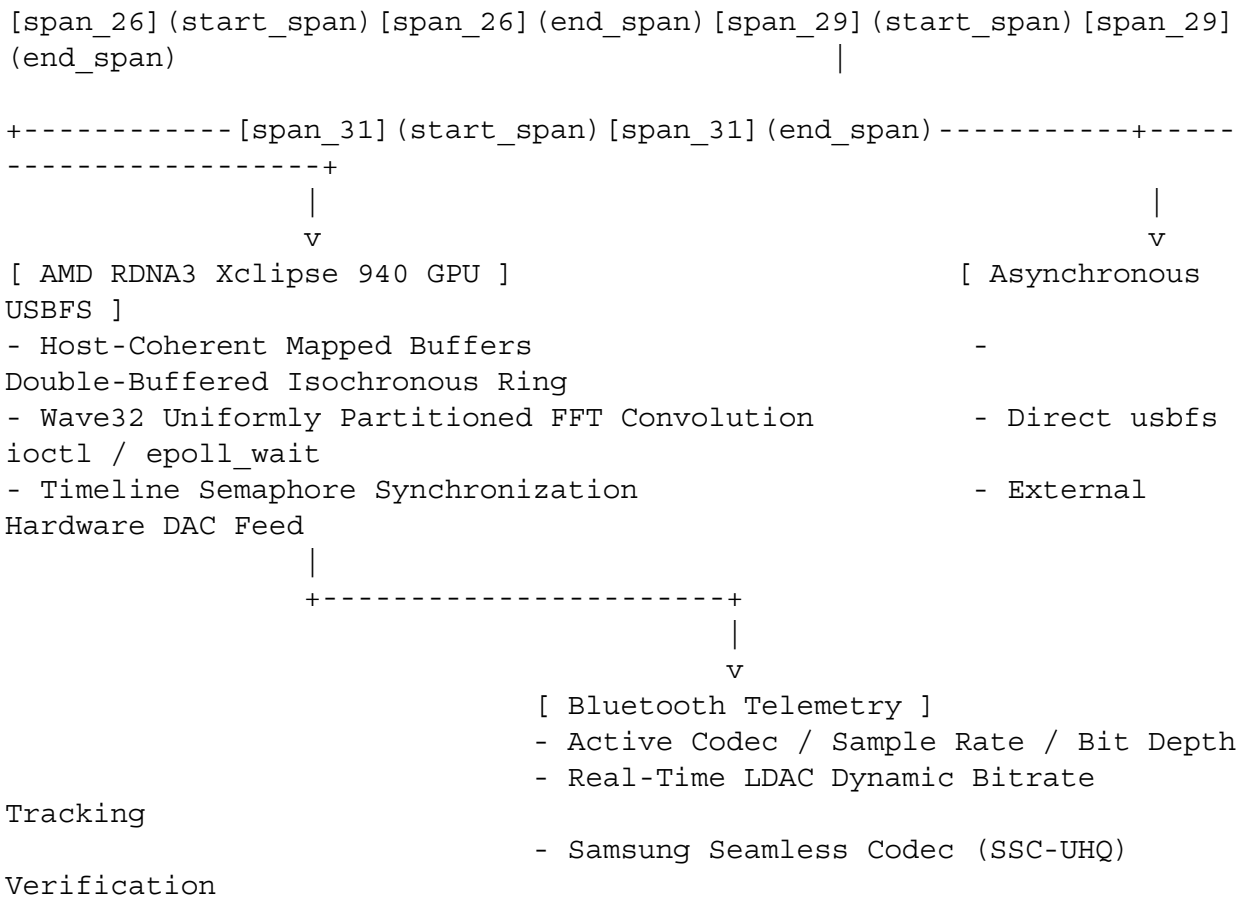
Architectural Synthesis and Hardware Integration

Implementing a deterministic, real-time audio engine on the Samsung Exynos 2400 requires careful coordination across system domains. At the CPU level, the engine pins its master processing thread to the dual high-clocked Cortex-A720 cores (Cores 7 and 8). This isolates the time-critical audio loop from scheduling stalls on the Cortex-A520 efficiency cores and avoids the thermal throttling associated with the Cortex-X4 core.

The direct usbfs audio pump handles Isochronous USB transfers asynchronously, while the Android Dynamic Performance Framework provides continuous timing feedback to the EAS governor. This maintains optimal CPU frequencies even under varying computational loads.

```
Data Processing Pipeline on the Samsung Exynos 2400:
[span_11](start_span)[span_11](end_span)
[ Linux Virtual Memory ]  -->  mmap(256MB) / madvise(MADV_WILLNEED) /
Proactive Page Touch
                                          |

[span_12](start_span)[span_12](end_span)    v
[ CPU Clusters 1 & 2 ]    -->  Cortex-A720 (Cores 4-6: usbfs / Cores
7-8: DSP & Ring Buffer)
```



To execute heavy digital signal processing within a sub-2 ms budget, the system offloads large convolutions to the AMD RDNA3-based Xclipse 940 GPU. Structuring the 131,072-tap filter as a Uniformly Partitioned Overlap-Save pipeline bounds algorithmic delay to the partition size (1.33\text{ ms} at 96 kHz). The compute shader aligns with the GPU's native Wave32 execution model to maximize ALU throughput across all 12 Compute Units.

Zero-copy, host-coherent buffers take advantage of the SoC's unified memory architecture, eliminating unnecessary data duplication between the CPU and GPU. Timeline semaphores coordinate GPU execution asynchronously, avoiding blocking calls on the audio callback thread. Memory management and signal integrity are handled via dedicated kernel and SIMD techniques. Operating within unrooted Android's 64\text{ KB} RLIMIT_MEMLOCK ceiling, the engine uses proactive page touching and kernel paging directives (MA[span_14](start_span)[span_14](end_span)DV_WILLNEED) to ensure zero-fault RAM playback for large audio buffers. In parallel, an ARM64 NEON routine continuously audits the audio stream for authentic 24-bit dynamics and flags fake upsamplings by analyzing ultrasonic spectral energy above 22.05 kHz.

Finally, a JNI-based Bluetooth telemetry bridge monitors active wireless links, extracting real-time LDAC bitrate steps (990\text{ / }660\text{ / }330\text{ kbps}) and verifying Samsung Seamless Codec connections. This integrated architecture delivers deterministic, low-latency audio processing while maintaining system stability and power efficiency under Samsung One UI.

Works cited

1. Samsung Exynos 2400: specs and benchmarks - NanoReview, https://nanoreview.net/en/soc/samsung-exynos-2400 2. Samsung Exynos 2400 Processor - Benchmarks and Specs, https://www.notebookcheck.net/Samsung-Exynos-2400-Processor-Benchmarks-and-Specs.808570.0.html 3. Handling sensitive data solely "in memory" likely impossible on most, https://github.com/eu-digital-identity-wallet/av-app-android-wallet-ui/issues/116 4. Performance Hint Manager - NDK - Android Developers, https://developer.android.com/ndk/reference/group/a-performance-hint 5. Performance Hint API | Android Open Source Project, https://source.android.com/docs/core/perf/performance-hint-api 6. Samsung Application Management, https://developer.samsung.com/mobile/app-management.html 7. چگونه Device Care سامسونگ را غیرفعال کنیم تا مصرف باتری کاهش, https://intotech.ir/39482 8. External Memory and Synchronization - Vulkan Documentation, https://docs.vulkan.org/guide/latest/extensions/external.html 9. Native Hardware Buffer | Android NDK, https://developer.android.com/ndk/reference/group/a-hardware-buffer 10. Layers & Extensions (Informative) :: Vulkan Documentation Project, https://docs.vulkan.org/spec/latest/appendices/extensions.html 11. matisiekpl/bluetooth-codec-change - GitHub, https://github.com/matisiekpl/bluetooth-codec-change 12. services/core/java/com/android/server/audio/BtHelper.java, https://android.googlesource.com/platform/frameworks/base/+/master/services/core/java/com/android/server/audio/BtHelper.java 13. Diff - platform/packages/modules/Bluetooth - Git at Google, https://android.googlesource.com/platform/packages/modules/Bluetooth/+/c55eb6ca1009267198d679835ecbcbdcc4d11621%5E2..c55eb6ca1009267198d679835ecbcbdcc4d11621/ 14. BluetoothCodecConfig.Builder.SetCodecType(Int32) Method, https://learn.microsoft.com/en-us/dotnet/api/android.bluetooth.bluetoothcodecconfig.builder.setcodectype?view=net-android-35.0 15. BluetoothCodecConfig.SourceCodecTypeInvalid 字段, https://learn.microsoft.com/zh-cn/dotnet/api/android.bluetooth.bluetoothcodecconfig.sourcecodectypeinvalid?view=net-android-37.0 16. BluetoothCodecType | API reference - Android Developers, https://developer.android.com/reference/android/bluetooth/BluetoothCodecType 17. …/BluetoothCodecConfig.java · Gerrit Code Review, https://review.carbonrom.org/c/CarbonROM/android_frameworks_base/+/13243/2/core/java/android/bluetooth/BluetoothCodecConfig.java 18. Diff - platform/frameworks/base - Git at Google, https://android.googlesource.com/platform/frameworks/base/+/69b6876c5804f83c195ffa67f09c7a5e41ee3fd5%5E2..69b6876c5804f83c195ffa67f09c7a5e41ee3fd5/ 19. Android蓝牙音频编解码器详解：AAC、LDAC与aptX适配优化原创, https://blog.csdn.net/weixin_29189363/article/details/166183081 20. BluetoothCodecType.CodecName Property (Android.Bluetooth), https://learn.microsoft.com/th-th/dotnet/api/android.bluetooth.bluetoothcodectype.codecname?view=net-android-37.0